Interactive Debugging and Steering of Multi-Agent AI Systems: A Re-implementation of AGDebugger with Error-Anchored Context Assembly

Nguyen Nhat Minh
nm.nguyen.2026@phdcs.smu.edu.sg
Singapore Management University
Singapore, Singapore

Do Duc Anh
doducanh.2026@phdcs.smu.edu.sg
Singapore Management University
Singapore, Singapore

ABSTRACT

Teams of large-language-model (LLM) agents fail in ways single-prompt tools cannot expose: one wrong message cascades through dozens of later ones. AGDebugger (Epperson et al., CHI 2025) answered this with an interactive debugger whose central interaction is reset-and-edit, restoring every agent to a checkpoint, editing the offending message and continuing in a forked session. We re-implemented its three features on AutoGen 0.7.5 with our own React front end, and added an extension the paper names as future work: error-anchored context assembly, in which one judge pass ranks suspect messages and a context panel gathers what the developer needs to edit them. We built a 29-task harness and found that a strong model never failed it in 97 runs while a smaller model produced natural failures, which became the study material. We replicated the paper's two-part study with SMU students, adding the extension as a third condition in the error-identification part. This report describes the system, the study method and its results, and compares them with the original findings.

KEYWORDS

multi-agent systems, LLM agents, debugging, interactive steering, replication

1 INTRODUCTION

Developers of multi-agent LLM systems watch teams of agents exchange 70 to 200 messages to answer one question, and when the answer is wrong they must find the message where it went wrong. Epperson et al. [1] interviewed five agent developers and distilled four design goals: understand the messages, send messages, reset and edit, and configure agents. Their tool, AGDebugger, has three features: a message history with sending and step-wise execution control, reset-and-edit with session forking, and an overview visualisation. In a study with 14 developers, reset-and-edit was the highest-rated feature (4.9 of 5), two of eight participants steered a failing run to the right answer, and the 24 edits fell into three strategies: adding specific instructions (14), simplifying (5) and modifying the plan (5). Participants spent about ten of fifteen minutes reading before their first edit, and the paper lists automatic error identification as future work.

Our project asks two questions. Can the three features be re-implemented faithfully on the current version of the agent framework, AutoGen [4], with real checkpoints rather than simulated ones? And does anchoring the developer's attention on a few suspect messages, with their context assembled, change how errors are found and fixed? We built the system (Section 2), a task harness that yields natural failing runs (Section 3), and ran the paper's study design with one added condition (Sections 4 to 6). The target users are developers of agent teams; their goal is to localise the failure and steer the team to a correct result by browsing and stepping a long history, sending messages, resetting to an earlier point and editing, with an overview of the branches.

2 SYSTEM

2.1 Scope and architecture

We re-implemented the three features (C1 history and sending, C2 reset-and-edit, C3 overview) and added E1, error-anchored context assembly. The agent-configuration panel (G4) was left out: it went unused in the paper's study and is read-only in the reference release. The reference code [5] was consulted only for its runtime hook; the implementation is our own, about 1,300 lines of Python and 800 of TypeScript.

Figure 1 shows the architecture. The team under debug is AutoGen's Magentic-One group chat [2]: an orchestrator that writes a fact sheet and a plan and then instructs a **Coder**, a **local code Executor** and a **WebSearcher** with search and fetch tools. The searcher reads either a 30-page fictional corpus (an observatory's website, a local newspaper and a near-namesake amateur observatory, with stale and conflicting figures) or live English Wikipedia through the MediaWiki API, both in chunks like a web surfer's viewport. The team runs on Claude Haiku 5.5 at low reasoning effort (Section 3 explains why).

Substrate. An AutoGen `InterventionHandler` intercepts every send, publish and RPC response, assigns a global timestamp and calls `runtime.save_state()` *before* the message is processed, so every agent's context can be restored at any timestamp. A recording (history, checkpoints, sessions, judge results) replays without model calls.

2.2 C1: history, sending, execution control

Each runtime message is a card with sender, recipient or topic, type and payload, tool calls included; a coloured bar marks the sending agent. The compose panel sends a broadcast, a direct request-to-speak or a new task. The queue panel shows the runtime's pending messages with *Run*, *Stop*, *Step* (process exactly one message) and *Drop*; a queued text message can be edited or removed before the agents see it, which the paper describes but its release did not implement.

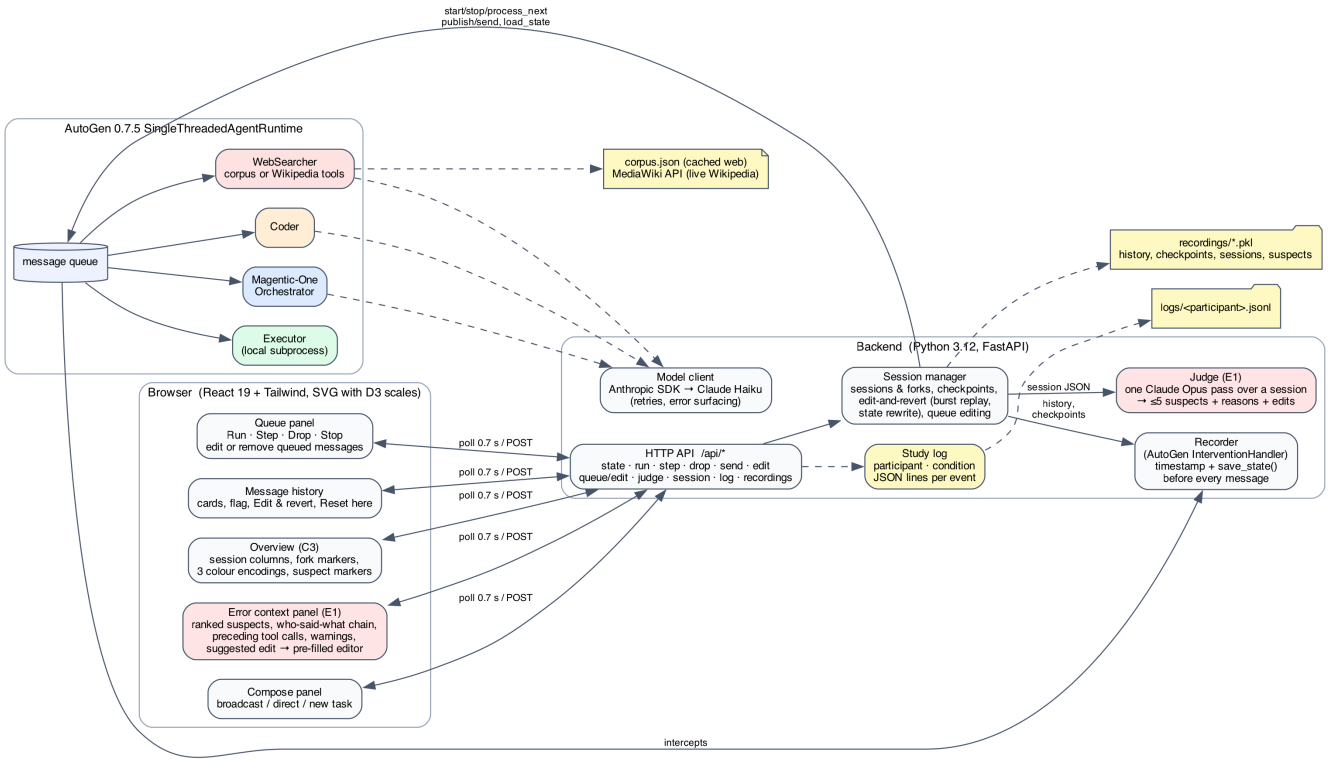


Figure 1: System architecture. The browser polls the backend; the session manager drives an AutoGen runtime whose every message is intercepted and checkpointed by the recorder; the judge and the study log are side components.

2.3 C2: reset-and-edit with session forking

Edit & revert and *Reset here* stop the runtime, snapshot the current session, restore the checkpoint taken before the chosen timestamp, re-publish the (edited) message and continue in a new session; earlier sessions stay viewable through tabs. Two details the paper leaves open had to be solved before a branch would continue. First, the checkpoint is taken when a message is dequeued, after its producer has already queued its whole burst, so the burst (the orchestrator's instruction plus its follow-up request-to-speak) is replayed, not just the edited message. Second, the sender's own memory of the message is already in its saved state, so the edit is rewritten into that state (the orchestrator's message thread and, for the plan message, its stored facts and plan); otherwise the orchestrator keeps acting on the text it remembers.

2.4 C3: overview visualisation

One column per session, one rectangle per message, laid out so that a fork starts at the row of the message it replaced; an amber marker shows the fork point. Colour encodes type, sender or recipient; legend entries filter; hover shows the message; a click scrolls the history to it. Suspects from E1 are outlined and numbered.

2.5 E1: error-anchored context assembly

E1 has four parts (Figure 2). (1) A *judge*: one call to Claude Opus 5.5 over the session's visible messages, the task and the expected answer (which the paper's participants were also given), returning up to five suspects, each with a category, a reason quoting the offending phrase, the evidence timestamps and a suggested edit. Suspects are stored with the recording, so study participants never wait for the model. (2) *Markers* in the overview and on the cards. (3) A *context panel*, computed from the history without a model call: the flagged message, the last three text messages exchanged with its sender or recipient or broadcast to the group, the tool calls since the previous orchestrator instruction, messages in that window that look like errors, and the judge's evidence. (4) *Edit this message & revert*, which opens the card's editor pre-filled with the original text and the suggestion. The developer still writes the edit; the judge never acts on the team.

2.6 Implementation notes

The framework's own Anthropic client crashed on the current SDK, so we wrote a `ChatCompletionClient` that replays reasoning blocks with tool calls, asks the API to drop reasoning blocks whose conversation prefix no longer matches (what a reset-and-edit produces), and renders earlier tool calls as text in the framework's tool-reflection turn. Model calls are retried; a failure is shown as a banner and the message is marked lost rather than ending the run; *Stop* keeps the queue so a branch can be resumed. The UI runs in three study conditions selected from the URL (`reduced` without reset and overview, `full`, `e1`, the cues of E1 existing only in the last), and every API action and UI event,

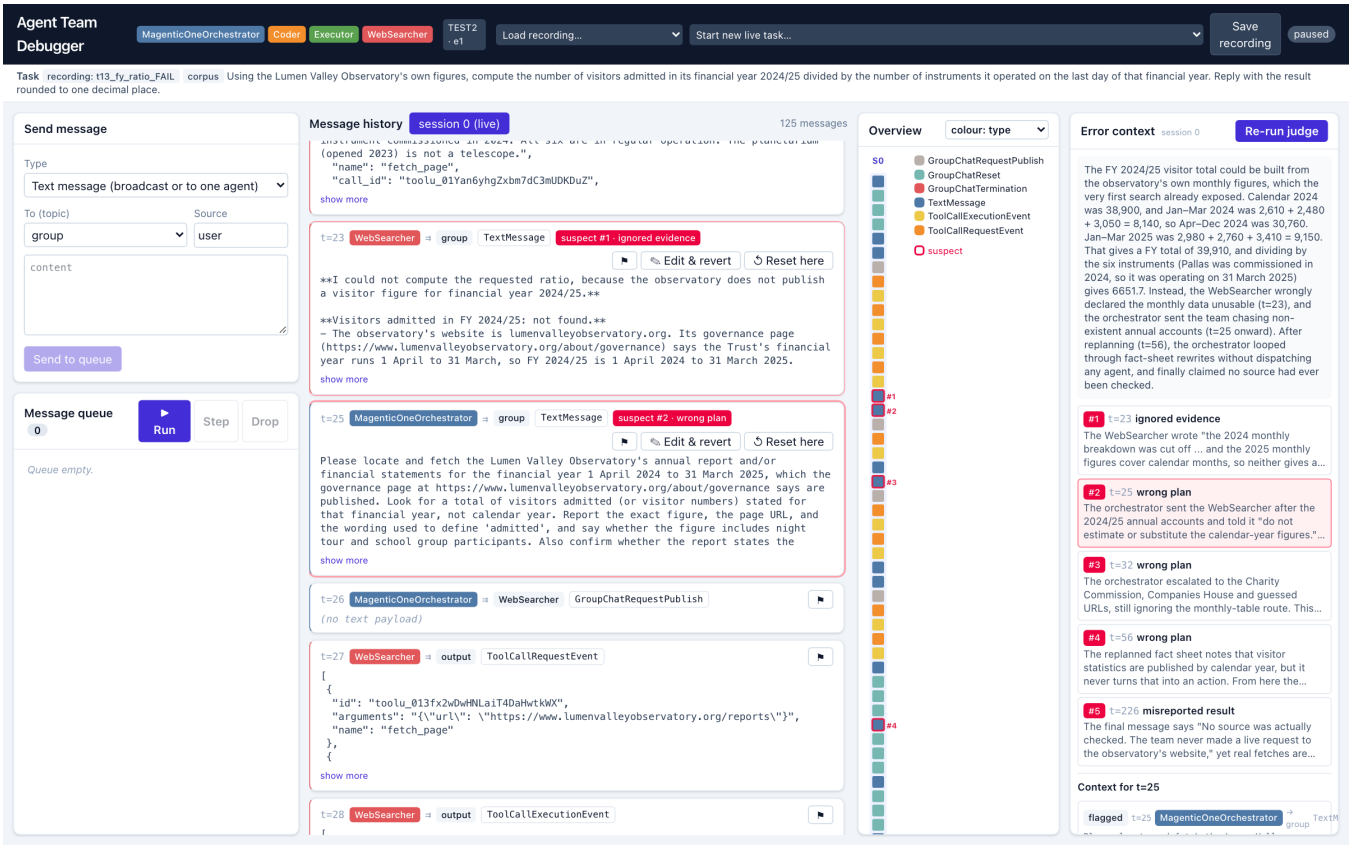


Figure 2: The E1 condition. Left: compose and queue. Centre: history with suspect badges. Right: overview with numbered suspect markers and the error context panel showing the judge's summary, five ranked suspects, and the assembled context for the selected one.

including a *flag* button that records an error judgement and the full text of every edit, is appended to a per-participant JSON-lines log.

3 HARNESS AND RECORDED FAILURES

The study needs recorded runs that fail naturally. We wrote 29 tasks with verified ground truth: 13 over the corpus (stale, conflicting or near-duplicate sources; date arithmetic; two coding tasks) and 16 GAIA-style [3] Wikipedia questions, five answerable only from tables. A harvest script runs each task repeatedly and grades the final answer by exact match.

Claude Opus 5.5 was correct in 97 of 97 runs: none of the paper's failure modes appeared. Claude Haiku 5.5 at low effort failed in 5 of 158 runs (two runs graded correct had hedged the figure as unverified, a weakness of exact matching that the study's human coding does not share). The failure mode is the same in every case: after one wrong turn the orchestrator's stall counter trips, it re-plans, and from then on rewrites its fact sheet 20 to 22 times without instructing any agent, until the turn limit (224 to 235 runtime messages, of which 119 to 134 are shown as cards once acknowledgements are hidden). Three recordings became the study material, with error lists coded by both authors (Table 1).

The judge's top two suspects coincide with our two primary errors on the first and third recordings. Editing suspect #2 of the first recording (the orchestrator's instruction at t=25, an *add specific instructions* edit) steered the same team to 6651.7 in 65 messages; editing the plan message at t=9 (a *modify plan* edit) did so in 23. Repeating the t=25 edit three times from the same checkpoint gave 3 of 3 branches that state 6651.7, although two of them first wandered into the re-planning loop and reached the answer only after 219 to 229 messages, hedged as "a derived estimate"; a plain reset at t=25 without the edit gave 0 of 3 (all three stalled again after 220 to 228 messages). For this one edit, then, the outcome difference is attributable to the edit, while the length of the repaired branch is not reproducible; this is the stochastic effect the paper calls "did my edit actually work?".

4 USER STUDY METHOD

We evaluated three things: whether users can localise the coded errors with the paper's features (C1 to C3) and whether E1 changes that; whether users can steer a failing run with reset-and-edit and which edit strategies they use; and how our ratings compare with the paper's.

Table 1: Study recordings: natural failures of the full Haiku team, with the error lists coded by both authors.

Recording	Task	Msgs	Errors	Primary errors
t13_fy_ratio_FAIL	FY visitors ÷ instruments (corpus)	228	5	searcher reports monthly tables as unusable (t=23); orchestrator forbids calendar-year figures (t=25)
t13_fy_ratio_FAIL_2	same task, second run	233	5	orchestrator rejects a correct derivation (t=36) and tells the Coder not to present 6651.7 (t=47)
wt2_mongolia_rank_FAIL	rank of Mongolia in a Wikipedia list	224	6	searcher reports "18th-largest" as unusable (t=36); orchestrator discounts it and never dispatches again (t=44)

Participants. We recruited [[FILL: N]] SMU computing students who had used LLM tools ([[FILL: programmes; agent-development experience from the pre-study questions, item 1 median]]), through course and lab mailing lists, with written consent; screen and audio were recorded and the tool's log kept under a participant number. No payment was given. The paper's participants were Microsoft researchers and interns, six of eight in Part 2 with agent-development experience; we report the same pre-study items for comparison.

Part 1: error identification (n = [[FILL: 6]], within-subjects). After a five-minute demo, each participant examined three recorded failing runs of matched length (Table 1), one per condition: *reduced* (history and sending only, the paper's baseline), *full* (the paper's feature set) and *full + E1*. Conditions and tasks followed a Latin square so that every condition met every task twice. Each task lasted 15 minutes; participants flagged messages in the tool and thought aloud. Unlike the paper's Part 1, participants were not asked to propose or make fixes, so this part measures localisation with and without the overview, reset and E1, not steering. Tasks A and B are two runs of the same question, so order and task effects are inspected before pooling. Measures: share of coded errors localised (flag or spoken, matched to the ground-truth list), whether a primary error was found, time to the first correct localisation, false localisations, confidence, and preference after the three tasks.

Part 2: interactive steering (n = [[FILL: 8]]). After the demo, each participant received the first recording with a summary of the errors found in Part 1 (as in the paper) and 30 minutes to steer the team to the correct answer with all features enabled, thinking aloud. Every branch continued live with real model calls. Measures: whether the correct answer was reached, number and type of edits (coded independently by both authors as add, simplify or modify, following the paper's scheme), whether the first edit targeted a judge suspect, time to the first edit, the paper's Appendix B Likert items plus one for the context panel, and the paper's exit interview. Part 2 is observational: every participant has E1, so it describes how the panel is used, not whether it helps. Comparisons with the paper are descriptive. [[FILL: if the fallback 12-participant single-study design was used, say so here.]]

5 RESULTS

> **To be completed after the study sessions (Weeks 12 to 15).** The tables and figures are generated by `prototype/study/analyze.py` and inserted at the marker below by `build/build.sh`; the prose around them is written once the data exist. Nothing here is filled by hand before then.

5.1 Overview of key findings

[[FILL: three to four sentences: Part 1 localisation by condition; Part 2 steering outcome; ratings versus paper; the one qualitative theme that dominated.]]

5.2 Part 1: error identification

[[FILL: one paragraph reading the table: what participants found in each condition, order and task effects (A versus B), with two quotes; whether E1's suspects were followed or disputed.]]

5.3 Part 2: steering

[[FILL: outcome count, edits per participant, edit-type distribution, first-edit target, time to first edit; two to three think-aloud examples of edits; what the ratings table shows.]]

6 DISCUSSION

Similarities with the paper. [[FILL after the study: e.g. whether backtrack-and-edit is again the top-rated feature; whether the add-specific-instructions strategy again dominates; whether steering to the exact answer remains hard (paper: 2 of 8).]] One similarity is already visible in the system work: the paper's observation that edits earlier in the conversation steer better than later ones matches our steering runs, where a plan edit at t=9 fixed the run in 23 messages and an instruction edit at t=25 in 65, while later messages are mostly re-plans with no downstream effect to change.

Differences. [[FILL: differences in localisation time and reading effort between conditions, and in ratings; the role of participant experience (students versus agent developers).]] A structural difference is the team under debug. The paper debugged GPT-4o-era runs of a five-agent Magentic-One team on GAIA. We found that a current strong model does not fail our 29 tasks at all, so natural failures had to come from a smaller model, and all of them share one failure mode (the orchestrator stalls into re-planning). This makes error identification more uniform than in the paper, where the Web Surfer's page handling was a major error source, and it shifts the debugging question from "which agent misread the page" to "which instruction made the orchestrator give up".

E1. [[FILL: whether ranked suspects shortened time to first localisation and first edit; whether participants trusted them; cases where the judge was wrong or where participants edited a non-suspect.]] The judge is one model call per session and reads the

expected answer, as the paper's participants did; it is instructed to describe what to change in a message and never to state or derive the answer, and it does not act on the team. A wrong suspect can still send a participant to a wrong edit, so the E1 condition tests the whole package (ranking, explanation, suggestion and context), not navigation alone.

Limitations. [[FILL: N]] participants, students rather than professional agent developers, one task family in Part 2, and recordings whose failures come from one model. Our corpus is synthetic (a fictional observatory), although the Wikipedia tasks are real. The "did my edit work" question remains open: the same edit from the same checkpoint can diverge, and our repeat-run check (Section 3) only bounds it for one edit.

7 CONCLUSION

We re-implemented AGDebugger's three features on real AutoGen checkpoints, documented two semantics the paper leaves open (burst replay and rewriting the edit into the sender's state), built a harness that yields natural failing runs, and added error-anchored context assembly as a third study condition. [[FILL: one sentence with the main study result.]] Code, recordings, study materials and the analysis script accompany this report.

REFERENCES

[1] Will Epperson, Gagan Bansal, Victor C. Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. ACM, Yokohama, Japan. https://doi.org/10.1145/3706598.3713581

[2] Adam Fourney, Gagan Bansal, Hussein Mozannar, et al. 2024. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. Technical Report MSR-TR-2024-47. Microsoft.

[3] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: A Benchmark for General AI Assistants. arXiv:2311.12983.

[4] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *COLM 2024*.

[5] Microsoft. 2025. AGDebugger reference implementation. https://github.com/microsoft/agdebugger (MIT licence).